

HASHING

PROBLEM 1

QUESTION:

DS	Session	Hashing	Question Information	Level 1	Challenge 91
Problem Description Everyone knows that some Pikachu despise becoming Raichu. (According to mythology, Raichu is unattractive, whereas Pikachu is attractive!) How do we track down these unique Pikachus who despise evolution? Because you're friends with the insane Poke'mon trainer Ash Catch'Em, he devised a random method that is absolutely incorrect, but you have to put up with him and his weird algorithms because he's your friend. He thinks if you are given N Pikachus in an array, $A_1, A_2 \dots A_N$, where each Pikachu is denoted by an integer. The total number of unique pairs (A_i, A_j) where $i < j$ is the number of Pikachus who hate evolution. Constraints: $1 \leq N \leq 2 * 10^5$ $1 \leq A_i \leq 10^9$ Input format: The first line will consist of a single integer N. The second line consists of N integers $A_1, A_2 \dots A_N$. Output format: Output the total number of unique pairs (A_i, A_j) that can be formed, which will also be the number of special Pikachus.					
✓ Logical Test Cases Test Case 1 INPUT (STDIN) 5 1 2 2 1 3 EXPECTED OUTPUT 6 Test Case 2 INPUT (STDIN) 7 1 4 1 2 2 1 3 EXPECTED OUTPUT 10					
✓ Mandatory Test Cases Test Case 1 KEYWORD for(i=0;i<n;i++) Test Case 2 KEYWORD if(arr[i]>max)					
✓ Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY 9 Test Case 2 TOKEN COUNT 330 Test Case 3 NLOC 55					

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int total_elements;
    if (scanf("%d", &total_elements) != 1) return 0;

    int items[total_elements];
    for (int i = 0; i < total_elements; i++) {
        scanf("%d", &items[i]);
    }

    static int pair_seen[1005][1005];
    int distinct_pair_count = 0;

    for (int i = 0; i < total_elements; i++) {
        for (int j = i + 1; j < total_elements; j++) {
            int val1 = items[i];
            int val2 = items[j];
            if (val1 <= 1000 && val2 <= 1000) {
                if (!pair_seen[val1][val2]) {
                    pair_seen[val1][val2] = 1;
                    distinct_pair_count++;
                }
            }
        }
    }

    printf("%d\n", distinct_pair_count);
    return 0;
}
```

OUTPUT:

```
5
1 2 2 1 3
6
```

```
7
1 4 1 2 2 1 3
10
```